



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Engine Architectures

Fachhochschule Oberösterreich

C++ OOP

Basics

C++ Functions

- **Bundle a set of actions** to be utilized various times
- **Name:** Unique identifier
- **Parameters:** A list of inputs that the function can use
- **Return value:** A particular type that will be returned as result, otherwise: void
- **Body:** Code necessary to execute the function (i.e., produce the return value)
- **Local variables:** variables needed for computation within the function
- **Signature:** name and combination of input parameters

Basics

C++ Procedures

- A function without a return type (i.e., void) is also called **procedure**
- Procedures can use the “return” statement to simply stop

```
void AddToLiquoreStoreCustomerList(string name, int age)
{
    if (age < 18)
        return;
    //some other stuff
}
```

Basics

C++ Inline Functions

- Used not to call the function, but copy the content into the code when called
- Why can this be useful?

```
inline int max(int a, int b)
{
    if (a >= b)
        return a;
    else
        return b;
}
```

No function overhead is generated, which reduces memory usage and increases performance!

Basics

C++ Predefined Function Parameters

- We can predefine **default values** for function parameters
- If a caller does not provide the arguments, they will be automatically set

```
bool checkDate(int day = 1, int month = 1, int year = 2000)
{
    return true;
}

bool checkDate(int day = 1, int month = 1)
{
    return true;
}

int main()
{
    checkDate();
}
```

Beware the signatures!
After a default parameter, all following
parameters must define default values, too!

Basics

C++ Functions

Browse to:

<https://onecompiler.com/cpp/43znmkuup>

Main.cpp

HGB_Pointers_2 

NEW

CPP ▾

RUN ▶



```
1  #include <iostream>
2  using namespace std;
3
4  int IncrementByThree(int number)
5  {
6      number = number+3;
7      return number;
8  }
9
10 int* IncrementByThreePtr(int number)
11 {
12     number = number+3;
13     return &number;
14 }
15
16 int main()
17 {
```

STDIN

Input for the program (Optional)

Output:

Result: 6

Basics

C++ Operator Functions

- Used to implement Operator Overloading
- Grammar: type operatorOP(type left, type right)

```
int operator+(int left, int right)
{
    return left - right;
}
```

Base type operators cannot be overloaded!

Basics

C++ Object-Oriented Programming

- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- **Abstraction**

Basics

C++ OOP – Encapsulation

- Bundling related **data and functions into classes**
- Hides the internal representation from outside (**“Information Hiding”**)

Basics

C++ OOP – Inheritance

- **Building a relationship** between classes and their members/functions
- Classes can be derived from other classes
- Used to form **hierarchies of classes** which share common properties or functionality
- **Class types:**
 - Base class (the root of the class hierarchy)
 - Superclass (the class the new class inherits from)
 - Subclass (the actual class which is derived from the superclass is a subclass)

Basics

C++ OOP – Abstraction

- **Appropriate hiding** of members, methods, and implementation details within a class
- **Access modifiers** in C++
 - Public: can be used from anywhere within the project/program
 - Private: can only be used within the class
 - Protected: can only be used within the class and derived classes
 - Classes themselves do not use access modifiers in C++

Basics

C++ OOP – Polymorphism

- Application of the same code to multiple classes and data types
- Implementing **concrete functionality in subclasses** but calling the function from the superclass
- The exact details of the functionality is provided by the subclass
- **Overriding**: a class may override an already implemented function from a superclass

Basics

C++ OOP – Constructors and Destructors

- A **constructor** is called when an object is **created**
- **Used to initialize objects**
 - Loading values
 - Define default values
 - Connect to hardware, etc.
- A **destructor** is called when the object goes **out of scope**
 - We will learn more about destructors later!

Basics

C++ Polymorphism

Polymorphism

What is the output of this program?

Super!

Sub!

(this is not polymorphism but only the use of class functionality)

```
class SuperClass
{
public:
    string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    string Print() { return "Sub!\n"; }
};

int main()
{
    SuperClass a;
    cout << a.Print();
    SubClass b;
    cout << b.Print();
}
```

Basics

C++ Polymorphism

Polymorphism

We add a function that works with any instance of the superclass "SuperClass"

```
class SuperClass
{
public:
    string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    string Print() { return "Sub!\n"; }
};
string PrintClass(SuperClass &c)
{ return c.Print(); }
int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```


Basics

C++ Polymorphism

Polymorphism

What is the output of this program?

Super!

Super!

```
class SuperClass
{
public:
    string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    string Print() { return "Sub!\n"; }
};
string PrintClass(SuperClass &c)
{ return c.Print(); }
int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Dynamic Casting

The function requests a SuperClass reference.

Calling the function casts a "SubClass" object to a "SuperClass" object

```
class SuperClass
{
public:
    string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    string Print() { return "Sub!\n"; }
};
string PrintClass(SuperClass &c)
{ return c.Print(); }
int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Binding

The binding (i.e., which function is called) is conducted at compile time (“early binding“)! 

```
class SuperClass
{
public:
    string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    string Print() { return "Sub!\n"; }
};
string PrintClass(SuperClass &c)
{ return c.Print(); }
int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Virtual Functions

We can mask a function as virtual to escape this problem...

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};

class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};

string PrintClass(SuperClass &c)
{ return c.Print(); }

int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

- So far (i.e., in Java, C#) we have learned that “**virtual**” is used to make a class abstract, i.e.,
- The function is not implemented
 - The class becomes abstract
 - No instance of the abstract class can be instantiated
 - Derived classes must implement the virtual function to allow instantiation
 - This concept is called “**pure virtual**”

- **Pure Virtual:** used to make a class abstract, i.e.,
 - A pure virtual function is not implemented (“=0”)
 - The class becomes abstract
 - No instance of the abstract class can be instantiated
 - Derived classes must implement the virtual function to allow instantiation

- **Virtual:** used to realize dynamic binding at runtime, i.e.,
 - Polymorphism: call the proper function depending on the type of an object
 - Virtual functions can be implemented
 - The class does not become abstract
 - Subclasses can override the virtual function to allow binding at runtime

Basics

C++ Polymorphism

Polymorphism: Virtual Functions

The compiler does not need to know which particular object is forwarded but due to the “virtual” keyword, it binds the function depending on the type at runtime (“late binding“)!

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};
string PrintClass(SuperClass &c)
{ return c.Print(); }
int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Virtual Functions

What is the output of this program?

Super!

Sub!

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};

class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};

string PrintClass(SuperClass &c)
{ return c.Print(); }

int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```


Basics

C++ Polymorphism

Polymorphism: Virtual Functions
When is this strategy useful?

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};

class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};

string PrintClass(SuperClass &c)
{ return c.Print(); }

int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Virtual Functions

Imagine an array with NPCs of different types, all derived from a base class called "StrategyGameObject" – a GameState class can have an array of these objects, including the player, NPCs, enemies, etc.

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};

class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};

string PrintClass(SuperClass &c)
{ return c.Print(); }

int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Virtual Functions

Here, the SuperClass parameter is forwarded via "Call by Reference"

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};

class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};

string PrintClass(SuperClass &c)
{ return c.Print(); }

int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Polymorphism

Polymorphism: Virtual Functions

What happens if we use Call by Value?

Early binding – the output is again

Super!

Super!

```
class SuperClass
{
public:
    virtual string Print() { return "Super!\n"; }
};
class SubClass : public SuperClass
{
public:
    virtual string Print() { return "Sub!\n"; }
};
string PrintClass(SuperClass c)
{ return c.Print(); }
int main()
{
    SuperClass a;
    cout << PrintClass(a);
    SubClass b;
    cout << PrintClass(b);
}
```

Basics

C++ Interfaces

Multiple Inheritance

We create a Superclass “Food” with a print function

```
class Food
{
public:
    virtual string Print() { return "Food!\n"; }
};
class Tomato : public Food
{
public:
    virtual string Print() { return "Tomato!\n"; }
};
class Ham : public Food
{
public:
    virtual string Print() { return "Ham!\n"; }
};
class Pizza : public Tomato, Ham
{
};

string PrintClass(Food &c)
{ return c.Print(); }
```

Basics

C++ Interfaces

Multiple Inheritance

We derive a class “Tomato” that overrides the print function

```
class Food
{
public:
    virtual string Print() { return "Food!\n"; }
};
class Tomato : public Food
{
public:
    virtual string Print() { return "Tomato!\n"; }
};
class Ham : public Food
{
public:
    virtual string Print() { return "Ham!\n"; }
};
class Pizza : public Tomato, Ham
{
};

string PrintClass(Food &c)
{ return c.Print(); }
```

Basics

C++ Interfaces

Multiple Inheritance

We derive a class “Ham” that overrides the print function, too

```
class Food
{
public:
    virtual string Print() { return "Food!\n"; }
};
class Tomato : public Food
{
public:
    virtual string Print() { return "Tomato!\n"; }
};
class Ham : public Food
{
public:
    virtual string Print() { return "Ham!\n"; }
};
class Pizza : public Tomato, Ham
{
};

string PrintClass(Food &c)
{ return c.Print(); }
```

Basics

C++ Interfaces

Multiple Inheritance

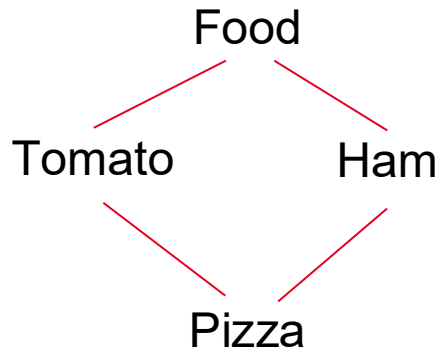
Finally, we implement a class “Pizza” that is derived from both “Tomato” and “Ham”

```
class Food
{
public:
    virtual string Print() { return "Food!\n"; }
};
class Tomato : public Food
{
public:
    virtual string Print() { return "Tomato!\n"; }
};
class Ham : public Food
{
public:
    virtual string Print() { return "Ham!\n"; }
};
class Pizza : public Tomato, Ham
{
};

string PrintClass(Food &c)
{ return c.Print(); }
```


Basics

C++ Interfaces



Multiple Inheritance

If we print a Pizza, which “print” function will be called?

```
class Food
{
public:
    virtual string Print() { return "Food!\n"; }
};
class Tomato : public Food
{
public:
    virtual string Print() { return "Tomato!\n"; }
};
class Ham : public Food
{
public:
    virtual string Print() { return "Ham!\n"; }
};
class Pizza : public Tomato, Ham
{
};

string PrintClass(Food &c)
{ return c.Print(); }
```

Basics

C++ Interfaces

Multiple Inheritance

At least, this is not possible and we will get an error...

```
string PrintClass(Food &c)
{ return c.Print(); }
```

```
int main()
{
    Pizza p;
    cout << p.Print();
}
```

std::string Tomato::Print()

[Online suchen](#)

""Pizza::Print"" ist mehrdeutig.

[Online suchen](#)

Basics

C++ Interfaces

One Solution:

Implement the print function in the "Pizza" class, too!

```
class Pizza : public Tomato, Ham
{
public:
    virtual string Print() {
        string pizzastring = "Pizza with ";
        string tomatostring = Tomato::Print();
        string hamstring = Ham::Print();
        return pizzastring + tomatostring + " and " + hamstring;
    }
};

string PrintClass(Food &c)
{ return c.Print(); }

int main()
{
    Pizza p;
    cout << p.Print();
}
```

Basics

C++ Interfaces

One Solution:

We can access both superclasses' functions

```
class Pizza : public Tomato, Ham
{
public:
    virtual string Print() {
        string pizzastring = "Pizza with ";
        string tomatostring = Tomato::Print();
        string hamstring = Ham::Print();
        return pizzastring + tomatostring + " and " + hamstring;
    }
};

string PrintClass(Food &c)
{ return c.Print(); }

int main()
{
    Pizza p;
    cout << p.Print();
}
```

- **Bundle functionalities that can be attached to various objects**
- **Do not have variables** but only function definitions
- An interface **provides an “ability”** to a class
- There are **no “instances” of interfaces**, thus:
 - Class names are nouns (Player, Food, Pizza, etc.)
 - **Interface names are adjectives** (Printable, Fightable, etc.) or have the prefix “I” (Ilog)
- **In C++**
 - There is no “interface” keyword
 - An interface is a **pure virtual class**
 - Consequence: Interface implementation is not enforced!

Basics

C++ Interfaces

Interface

We create an interface class that contains the print functionality

```
class IPrintable
{
public:
    virtual string Print() = 0;
};

class Food : public IPrintable
{
public: virtual string Print() { return "Food"; }
};

class Tomato : public Food, IPrintable
{
public: virtual string Print() { return "Tomato!\n"; }
};

class Ham : public Food, IPrintable
{
public: virtual string Print() { return "Ham!\n"; }
};

class Pizza : public Tomato, Ham, IPrintable
{
public:
    virtual string Print() { return "Pizza with Ham and Tomato!"; }
};
```

Basics

C++ Interfaces

Interface

Potential other classes implement the interface when including it in their class definition

```
class IPrintable
{
public:
    virtual string Print() = 0;
};

class Food : public IPrintable
{
public: virtual string Print() { return "Food"; }
};

class Tomato : public Food, IPrintable
{
public: virtual string Print() { return "Tomato!\n"; }
};

class Ham : public Food, IPrintable
{
public: virtual string Print() { return "Ham!\n"; }
};

class Pizza : public Tomato, Ham, IPrintable
{
public:
    virtual string Print() { return "Pizza with Ham and Tomato!"; }
};
```

Basics

C++ Interfaces

Interface

What is the benefit of using interfaces?

```
class IPrintable
{
public:
    virtual string Print() = 0;
};

class Food : public IPrintable
{
public: virtual string Print() { return "Food"; }
};

class Tomato : public Food, IPrintable
{
public: virtual string Print() { return "Tomato!\n"; }
};

class Ham : public Food, IPrintable
{
public: virtual string Print() { return "Ham!\n"; }
};

class Pizza : public Tomato, Ham, IPrintable
{
public:
    virtual string Print() { return "Pizza with Ham and Tomato!"; }
};
```


Basics

C++ Interfaces

Interface

What is the benefit of using interfaces?

Any class can use it – for example, the Player class can also provide a print functionality but is no “Food”

```
class IPrintable
{
public:
    virtual string Print() = 0;
};

class Food : public IPrintable
{
public: virtual string Print() { return "Food"; }
};

class Tomato : public Food, IPrintable
{
public: virtual string Print() { return "Tomato!\n"; }
};

class Ham : public Food, IPrintable
{
public: virtual string Print() { return "Ham!\n"; }
};

class Pizza : public Tomato, Ham, IPrintable
{
public:
    virtual string Print() { return "Pizza with Ham and Tomato!"; }
};
```

→ Object Oriented Programming

- Allows to build hierarchies of classes via inheritance
- Subclasses can use functionality of the superclass(es)
- Virtual functions should be used always when the functions will be overridden
- Avoid multiple inheritance, rather use interfaces
- Subclasses can override functions but still call the superclass method
- Use abstraction whenever possible (i.e., hiding code) to reduce the chance for bugs

→ Object oriented programming may sound complex but is still the **most sophisticated way of building good software!**

- Imagine the following situation
- We have a function Max that returns the larger one of two integers
- What if we want to find the Max of two float values?

```
int Max(int a, int b)
{
    return (a >= b) ? a : b;
}
```

```
int main()
{
    ...
}
```

Basics

C++ Templates

- Imagine the following situation
- We have a function Max that returns the larger one of two integers
- What if we want to find the Max of two float values?
- Implement the function from scratch for each datatype...
- Or, use **Templates**

```
int Max(int a, int b)
{
    return (a >= b) ? a : b;
}
```

```
float Max(float a, float b)
{
    return (a >= b) ? a : b;
}
```

```
int main()
{
    ...
}
```

Basics

C++ Templates

- We can create a template for a class that we call “NumberContainer”
- This class **can work with arbitrary types** (not only numbers)
- The concrete type will then be given when needed

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
};

int main()
{
    NumberContainer<int> num;
    num.setData(4.4);

    NumberContainer<float> num1;
    num1.setData(4.4);
    cout << num.getData() << endl;
    cout << num1.getData() << endl;
}
```

Basics

C++ Templates

Template

We define a template where the type will be substituted with the term “T”

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
};

int main()
{
    NumberContainer<int> num;
    num.setData(4.4);

    NumberContainer<float> num1;
    num1.setData(4.4);
    cout << num.getData() << endl;
    cout << num1.getData() << endl;
}
```

Basics

C++ Templates

Template

We define a private variable of yet unknown type "T" called "data"

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
};

int main()
{
    NumberContainer<int> num;
    num.setData(4.4);

    NumberContainer<float> num1;
    num1.setData(4.4);
    cout << num.getData() << endl;
    cout << num1.getData() << endl;
}
```

Basics

C++ Templates

Template

We implement the getter/setter for the private variable

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
};

int main()
{
    NumberContainer<int> num;
    num.setData(4.4);

    NumberContainer<float> num1;
    num1.setData(4.4);
    cout << num.getData() << endl;
    cout << num1.getData() << endl;
}
```


Basics

C++ Templates

Template

Now, we can utilize the template by using it with a concrete type

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
};

int main()
{
    NumberContainer<int> num;
    num.setData(4.4);

    NumberContainer<float> num1;
    num1.setData(4.4);
    cout << num.getData() << endl;
    cout << num1.getData() << endl;
}
```

Basics

C++ Templates

Template

However, to provide the Max() functionality, we also have to implement a template function

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
    bool operator>=(NumberContainer<T>& right)
    {
        return (data >= right.getData()) ? true : false;
    }
};

template <typename T>
T Max(T a, T b)
{
    return (a >= b) ? a : b;
}
```

Basics

C++ Templates

Template

To allow using the `>=` operator (as defined in the Max function), we have to overload the corresponding operator

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
    bool operator>=(NumberContainer<T>& right)
    {
        return (data >= right.getData()) ? true : false;
    }
};

template <typename T>
T& Max(T& a, T& b)
{
    return (a >= b) ? a : b;
}
```

Basics

C++ Templates

Template

Would there be an alternative?

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
    bool operator>=(NumberContainer<T>& right)
    {
        return (data >= right.getData()) ? true : false;
    }
};

template <typename T>
T& Max(T& a, T& b)
{
    return (a >= b) ? a : b;
}
```

Basics

C++ Templates

Template

Would there be an alternative?

We could also return in the Max function:
(a.getData()>=b.getData())? a:b;

```
template <typename T>
class NumberContainer
{
private:
    T data;
public:
    T getData() { return data; }
    void setData(T data) { this->data = data; }
    bool operator>=(NumberContainer<T>& right)
    {
        return (data >= right.getData()) ? true : false;
    }
};

template <typename T>
T& Max(T& a, T& b)
{
    return (a >= b) ? a : b;
}
```

Basics

C++ Enumerations

- Distinct type limited to a range of particular values
- Includes a **list of named constants** (“enumerators”)
- An enum has an integral type (frequently integer)
 - The type can be explicitly given
 - But also omitted

```
enum PlayerState {idle, running, walking, fighting};
```

```
int main()  
{  
    PlayerState p = idle;  
}
```

Basics

C++ Enumerations

```
enum PlayerState {idle, running, walking, fighting};
```

```
int main()  
{  
    PlayerState p = idle;  
}
```

Enum

Directly using the enum names could be confusing in larger projects, so we use enum class

Basics

C++ Enumerations

```
enum class PlayerState {idle, running, walking, fighting};
```

```
int main()  
{  
    PlayerState p = PlayerState::idle;  
}
```

Enum

Enum class allows us to use the class name as prefix – anyone reading the code knows now that “idle” is a “PlayerState”

Basics

C++ Enumerations

```
enum class PlayerState : int {idle, running, walking, fighting};
```

```
int main()  
{  
    PlayerState p = PlayerState::idle;  
}
```

Enum

We can also specify a data type which is used in the underlying logic

Basics

C++ Enumerations

```
enum class PlayerState : int {idle=1, running=2, walking=3, fighting=4};
```

```
int main()
{
    PlayerState p = PlayerState::idle;
    cout << p << endl;
}
```

Enum

We can define particular enum values for each manifestation

Basics

C++ Enumerations

```
enum class PlayerState : int {idle=1, running=2, walking=3, fighting=4};
```

```
int main()  
{  
    PlayerState p = PlayerState::idle;  
    cout << p << endl;  
}
```

Enum

However, we cannot simply output enums as strings – why not?

Basics

C++ Enumerations

```
enum class PlayerState : int {idle=1, running=2, walking=3, fighting=4};
```

```
int main()  
{  
    PlayerState p = PlayerState::idle;  
    cout << p << endl;  
}
```

Enum

However, we cannot simply output enums as strings – why not?

PlayerState is not overriding <<

Basics

C++ Enumerations

Enum

We implement a function that returns the particular enum value as string

```
string PlayerStateToString(PlayerState p)
{
    switch (p) {
        case PlayerState::idle: return "idle";
        case PlayerState::running: return "running";
        case PlayerState::walking: return "walking";
        case PlayerState::fighting: return "fighting";
    }
}

int main()
{
    PlayerState p = PlayerState::idle;
    cout << PlayerStateToString(p) << endl;
}
```

Game Engine Architectures



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at